ASCL
Automated Systems & Computing Lab

ACKNOWLEDGMENTS

This Training is supported by a research grant funded by the Research, Development, and Innovation Authority (RDIA), Kingdom of Saudi Arabia, with grant number 13382-psu-2023- PSNU-R-3-1-EI.

Prof. Ahmad Taher Azar

Full Professor, Leader of Automated Systems & Computing Lab (ASCL)

Logistic Regression

1. The Intuition: Why not Linear Regression?

We want to classify data into categories. Computers understand numbers, so we assign labels:

How we assign labels (Encoding):

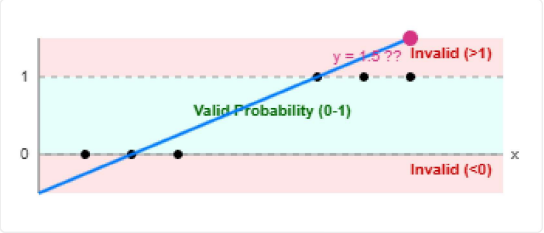
- **2 Classes (Binary):** 0 = "No/Negative", 1 = "Yes/Positive" (e.g., Spam vs. Not Spam)
- **3 Classes:** 0 = "Cat", 1 = "Dog", 2 = "Bird"
- **4 Classes:** 0 = "Winter", 1 = "Spring", 2 = "Summer", 3 = "Fall"
- **N Classes:** 0, 1, 2, ..., N-1

For Logistic Regression, we start with the simplest case: **Binary Classification (0 or 1)**.

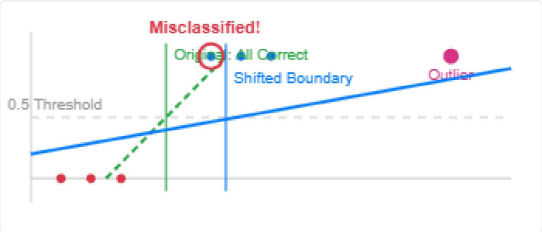
The Problem with Lines (Linear Regression)

If we try to fit a straight line $h_w(x) = W^T x + b$ through binary data:

- **Unbounded Output:** The equation $y = mx + c$ extends infinitely in both directions. For a large x, y could be 5, 10, or 100. Since probability must be between 0 and 1, a value of 5 (500%) makes no sense.



- **Sensitive to Outliers:** Linear regression tries to minimize the Mean Squared Error (MSE): $\frac{1}{m}\sum(h_w(x^{(i)}) - y^{(i)})^2$. A single outlier far away (e.g., at $x = 100$) will have a huge error, forcing the line to tilt significantly to reduce it. This shift moves the decision boundary (where $y = 0.5$), often causing misclassification of normal data points.



How Classification Works: We pick a **Threshold of 0.5** (dashed line).

- If predicted value > 0.5, we predict **Class 1 (Blue).**
- If predicted value < 0.5, we predict **Class 0 (Red).**

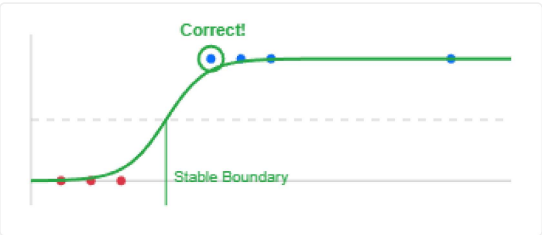
The **Decision Boundary** is where the line crosses 0.5.
Problem: The outlier pulls the blue line down, shifting the boundary to the right (from x=4.5 to x=6.5). Now, the blue point at **x=6** is below the line (predicted < 0.5), so it is **misclassified** as Red.

The Logistic Solution

Logistic Regression wraps the line in a function that bends it into an **S-shape**.

- **Bounded [0, 1]:** The Sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$ forces all outputs to stay strictly between 0 and 1. Even if x is huge, y never exceeds 1.

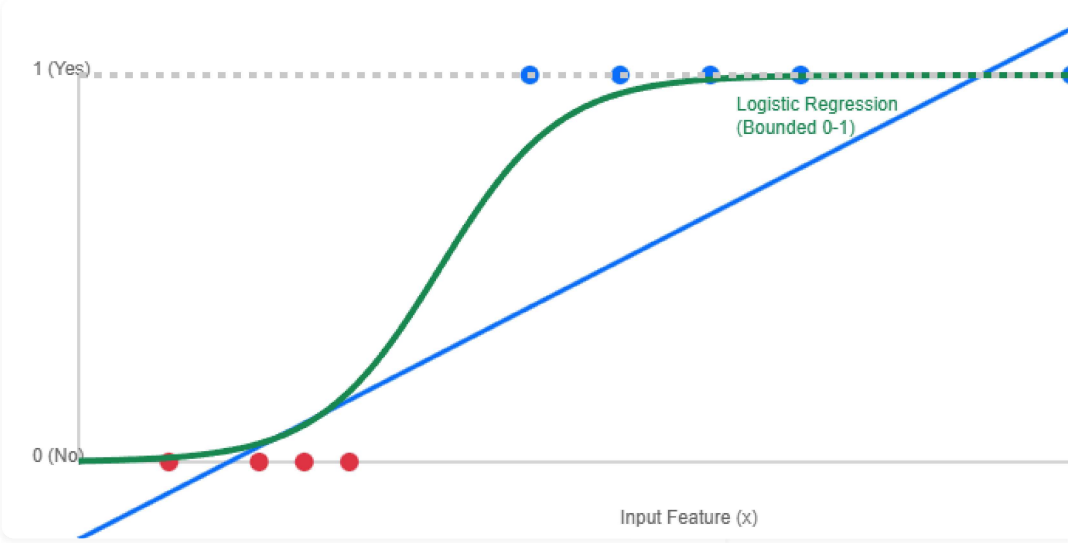
- **Robust to Outliers:** The error function (Log Loss) penalizes wrong confidence, not just distance. Far-away outliers are already "confident" (close to 1), so the curve doesn't need to shift much to accommodate them.



Stable Boundary: Even with the outlier at **x=14**, the decision boundary (0.5 crossing) stays near **x=4.5.**
Result: The point at **x=6** is still correctly classified as Blue (Class 1).

Visual Proof: Linear vs. Logistic Fit

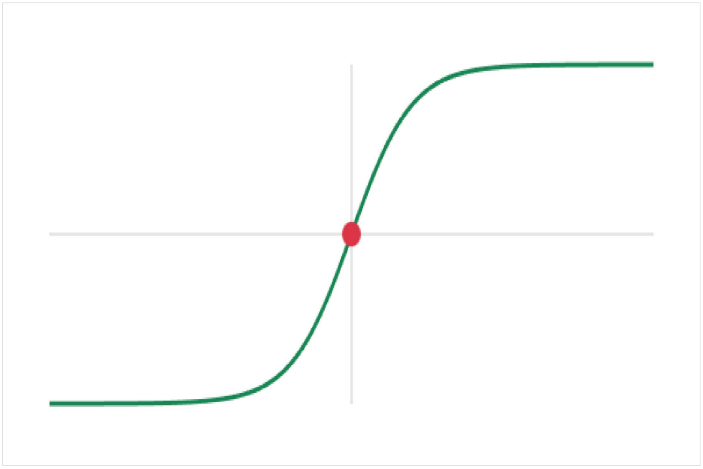
Notice how the Blue Line (Linear) shoots off the chart, while the Green Curve (Logistic) stays perfectly within the valid range.



2. Visualization: The Sigmoid Curve

Move the slider to see how the input value z transforms into a probability $P(y = 1)$.

Clear Reset



Input Value (z): 0

Probability: **0.50**

3. The Math: The Sigmoid Function

We model the probability of the positive class ($Y=1$) as:

The Sigmoid Function

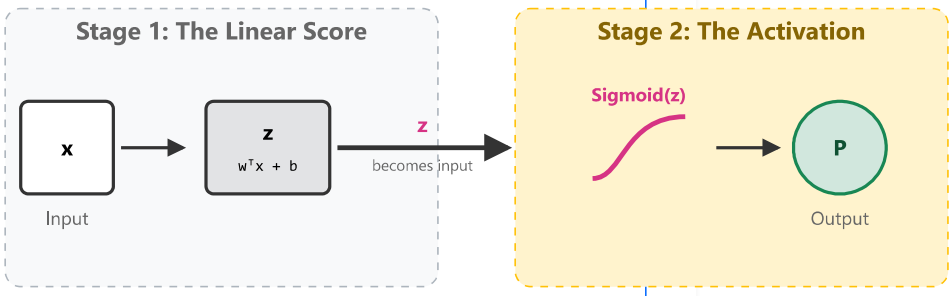
$$P(Y = 1 \mid X = x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$z = w^\top x + b$$

> ***z** is just our familiar linear equation (like in linear regression). We pass it through the σ function to force it between 0 and 1.*

Visualizing the Flow: Two Stages

Stage 1: Compute Linear Score (z) → Stage 2: Convert to Probability (P)



Log-Likelihood Loss

$$\ell(w, b) = \sum_{i=1}^{n} \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

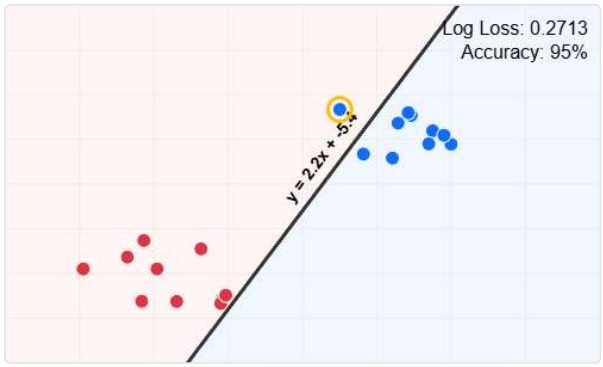
Understanding the Variables:

- $\ell(w, b)$**:** The total Log-Likelihood (how "good" our model is). We want to maximize this (or minimize the negative cost).
- $\sum_{i=1}^{n}$ **(Summation):** We loop through **all** n **training examples**. We sum up the score for every single data point because we want the model to work well for the *entire* dataset, not just one example.
- n**:** The total number of samples (e.g., 100 students).
- y_i**:** The **Actual Label** for the i-th student (0 or 1).
- $\hat{y}_i$**:** The **Predicted Probability** for the i-th student (output of the sigmoid function).

> *We don't use "Squared Error" here. We use "Cross-Entropy" (Log-Likelihood), which heavily punishes confident wrong predictions (e.g., predicting 99% probability for an event that doesn't happen).*

Interactive: Manual Training (Tune the Boundary)

Can you find the best line to separate the Red (0) and Blue (1) points? Adjust the sliders to lower the Loss.



Slope (w_1)

2.2

Intercept (b)

-5.4

☑ Flip Class Direction (Red Above?)

Check this if your line separates the points but Accuracy is near 0%.

Current Loss

0.2713

Accuracy: 95%

Goal: Get close to 0

Gradient Descent (Chain Rule)

To update weights, we calculate the gradient of the loss with respect to w and b using the Chain Rule:

$$\frac{\partial \ell}{\partial w} = \frac{\partial \ell}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

> *Breaking it down:*
>
> ***1. Derivative of Loss w.r.t Prediction ($\frac{\partial \ell}{\partial \hat{y}}$):***

Recall Loss $\ell = -[y \ln(\hat{y}) + (1-y)\ln(1-\hat{y})]$.
Differentiating w.r.t $\hat{y}$:

$$\frac{\partial \ell}{\partial \hat{y}} = -\left[\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}}\right] = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})}$$

2. Derivative of Sigmoid w.r.t z ($\frac{\partial \hat{y}}{\partial z}$***):***

Since $\hat{y} = \sigma(z) = (1+e^{-z})^{-1}$.
Using the chain rule:

$$\frac{d}{dz}(1+e^{-z})^{-1} = -1(1+e^{-z})^{-2} \cdot (-e^{-z})$$

$$= \frac{e^{-z}}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}}$$

$$= \sigma(z) \cdot (1-\sigma(z)) = \hat{y}(1-\hat{y})$$

3. Derivative of Linear Part w.r.t w ($\frac{\partial z}{\partial w}$***):***

Since $z = w \cdot x + b$*, the rate of change of* z *as* w *changes is simply* x.

$$\frac{\partial z}{\partial w} = x$$

- $\frac{\partial \ell}{\partial \hat{y}} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})}$ *(Rate of change of loss)*
- $\frac{\partial \hat{y}}{\partial z} = \hat{y}(1-\hat{y})$ *(Slope of Sigmoid)*
- $\frac{\partial z}{\partial w} = x$ *(Input value)*

Combining these gives the clean update rule:

$$\frac{\partial \ell}{\partial w} = (\hat{y}-y) \cdot x$$

$$\frac{\partial \ell}{\partial b} = (\hat{y}-y)$$

Finally, we update the parameters (weights and bias) using the Learning Rate (α):

$$w_{new} = w_{old} - \alpha \cdot \frac{\partial \ell}{\partial w}$$

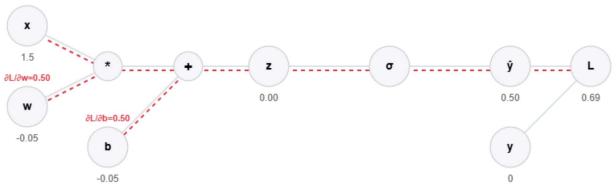
$$b_{new} = b_{old} - \alpha \cdot \frac{\partial \ell}{\partial b}$$

Interactive Computation Graph: Forward & Backward Pass

Click "Train Step" to visualize the flow of data (Forward) and gradients (Backward). We cycle through 10 training examples (5 from Class 0, 5 from Class 1).

Show Dataset Values

#	x (Input)	y (Label)	#	x (Input)	y (Label)
1	1.0	0	6	11.0	1
2	1.5	0	7	11.5	1
3	2.0	0	8	12.0	1
4	2.5	0	9	12.5	1
5	3.0	0	10	13.0	1



Epoch: 1
Loss: 0.6948

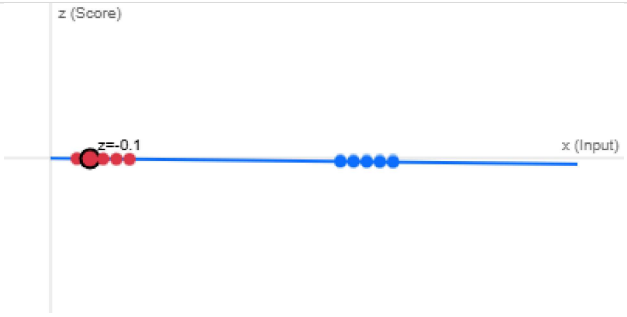
Forward

Backward

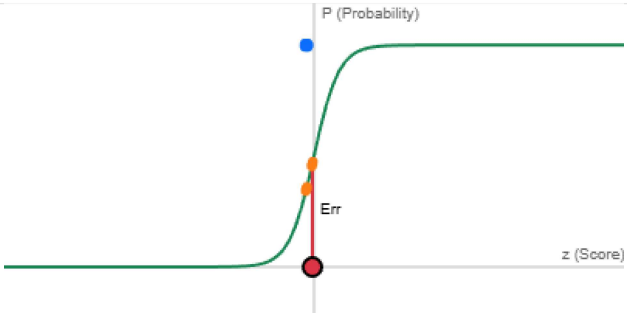
Reset Weights

Ready. Click 'Forward' to calculate predictions.

Linear Step: z = w·x + b



Activation Step: ŷ = σ(z)



4. Python Implementation

Using `LogisticRegression` from scikit-learn on the Iris dataset.

```python
from sklearn.linear_model import LogisticRegressic
from sklearn.model_selection import train_test_spl
from sklearn.metrics import accuracy_score, classi
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
import numpy as np

def classification_demo_logistic():
    """
    Logistic Regression demo on Iris dataset.
    """
    # 1. Load Data
    iris = load_iris()
    X = iris.data
    y = iris.target
    target_names = iris.target_names
    feature_names = iris.feature_names

    # Split data
    X_train, X_test, y_train, y_test = train_test_

    # 2. Scale Features (Helps optimization conver
    scaler = StandardScaler()
    X_train_s = scaler.fit_transform(X_train)
    X_test_s = scaler.transform(X_test)

    # 3. Train Model (All Features)
    logreg = LogisticRegression(max_iter=200)
    logreg.fit(X_train_s, y_train)

    # 4. Predict
    y_pred = logreg.predict(X_test_s)

    # 5. Evaluate
    print("Logistic Regression accuracy:", accurac
    print("\nClassification report:")
    print(classification_report(y_test, y_pred, ta

    # 6. Visualization (2D Projection)
    print("\nPlotting decision boundaries (using f

    # Retrain on just the first two features for 2
    X_2d = X[:, :2]  # Sepal length, Sepal width

    # Train simplified model for visualization
    logreg_2d = LogisticRegression(max_iter=200)
    logreg_2d.fit(X_2d, y)

    # Create plotting meshgrid
    x_min, x_max = X_2d[:, 0].min() - 0.5, X_2d[:,
    y_min, y_max = X_2d[:, 1].min() - 0.5, X_2d[:,
```

```
    h = 0.02  # Step size
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h
    # Predict for each point in meshgrid
    Z = logreg_2d.predict(np.c_[xx.ravel(), yy.rav
    Z = Z.reshape(xx.shape)
    # Plot
    plt.figure(figsize=(8, 6))
    plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm
    scatter = plt.scatter(X_2d[:, 0], X_2d[:, 1],
    plt.xlabel(feature_names[0])
    plt.ylabel(feature_names[1])
    plt.title('Logistic Regression Decision Bounda
    plt.legend(handles=scatter.legend_elements()[0
    plt.show()

classification_demo_logistic()
```

5. Step-by-Step Math: Logistic Regression Update

Let's trace one step of the algorithm with a tiny dataset to see how weights get updated.

The Dataset (1 Feature)

We have 4 examples. We want to classify them based on feature x.

Example	Feature (x)	True Label (y)	Note
1	1.0	0	Small x, Class 0
2	2.0	0	Small x, Class 0
3	4.0	1	Large x, Class 1
4	5.0	1	Large x, Class 1

Initialization

We start with random weights. Let's pick simple ones:

- Weight $w = 0.0$
- Bias $b = 0.0$
- Learning Rate $\eta = 0.1$

Processing Example 3 ($x = 4, y = 1$)

Let's see what happens when the model sees Example 3. It *should* predict 1.

1. Forward Pass (Prediction)

First, compute the linear score (logit) z:

$$z = w \cdot x + b = 0.0 \cdot 4.0 + 0.0 = 0.0$$

Next, convert to probability using Sigmoid $\sigma(z)$:

$$\hat{y} = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{0}} = \frac{1}{2} = 0.5$$

The model is unsure (50/50 chance).

2. Calculate Error (Gradient)

How wrong was it? We want $y = 1$, but got $\hat{y} = 0.5$.

$$\text{Error} = (\hat{y} - y) = 0.5 - 1.0 = -0.5$$

3. Backward Pass (Update Weights)

We nudge the weights opposite to the error gradient.

Update Bias b:

$$b_{new} = b - \eta \cdot (\hat{y} - y)$$

$$b_{new} = 0.0 - 0.1 \cdot (-0.5) = 0.0 + 0.05 = \mathbf{0.05}$$

Update Weight w:

$$w_{new} = w - \eta \cdot (\hat{y} - y) \cdot x$$

$$w_{new} = 0.0 - 0.1 \cdot (-0.5) \cdot 4.0$$

$$w_{new} = 0.0 + 0.2 = \mathbf{0.2}$$

Result

After seeing just this one positive example, the weight w increased to 0.2.
If we check Example 4 ($x = 5$) now:

$$z = 0.2 \cdot 5 + 0.05 = 1.05$$

$$\hat{y} = \frac{1}{1 + e^{-1.05}} \approx 0.74$$

The probability jumped from 0.5 to 74%! The model learned that larger x means Class 1.

Back to Dashboard | Next: Support Vector Machines →